

# Explicacao da Logica de Simulacao

Frontend conectado ao backend: sidebar, pagina, lib de simulacao, historico e tabela de resultados

Projeto: PipeAPI Dashboard

Documento atualizado com a persistencia das chamadas em memoria.

## 1. Objetivo da mudanca

A mudanca separa a responsabilidade de interface da responsabilidade de regra de negocio. Antes, a sidebar conhecia os endpoints, montava payloads e chamava a API. Agora, ela apenas coleta os valores do usuario e avisa a pagina que uma simulacao deve ser executada.

Com isso, a logica de simulacao ficou centralizada em um arquivo da lib, tornando o codigo mais facil de testar, evoluir e reutilizar em outros componentes.

## 2. Arquivos envolvidos

| Arquivo                                        | Responsabilidade                                                                               |
|------------------------------------------------|------------------------------------------------------------------------------------------------|
| src/components/dashboard/dashboard-sidebar.tsx | Renderiza a UI da sidebar, controla os inputs e emite os valores escolhidos pelo usuario. A se |
| src/components/dashboard/dashboard-page.tsx    | Orquestra a simulacao, controla loading/erro e guarda o historico das chamadas salvas.         |
| src/lib/simulation.ts                          | Monta o payload correto, chama a API certa e define os tipos compartilhados, incluindo Save    |
| src/lib/payloads.ts                            | Forneca os payloads padrao usados como base para cada endpoint.                                |
| src/lib/api.ts                                 | Contem as funcoes HTTP que fazem POST para o backend local.                                    |
| src/components/dashboard/dashboard-table.tsx   | Recebe a lista de resultados salvos do modulo/modo atual e renderiza todas as execucoes er     |

## 3. Fluxo geral

1. O usuario escolhe o Simulation Module na sidebar.
2. O usuario escolhe o Query Mode: Table ou Plot Data.
3. A sidebar renderiza os campos de Entity Parameters e os campos X/Y do modo escolhido.
4. O usuario altera os valores e clica em Run Simulation.
5. A sidebar envia module, queryMode e values para o DashboardPage.
6. O DashboardPage chama runSimulation(formValues).
7. A lib simulation.ts monta o payload com base no payloadDefault.
8. A lib simulation.ts chama a funcao correta de api.ts.
9. O resultado volta para o DashboardPage.
10. O DashboardPage salva a chamada no historico do modulo/modo correto.
11. O DashboardPage repassa a lista salva para DashboardTable, que exibe os dados.

### Representacao resumida

```
Sidebar -> DashboardPage -> runSimulation() -> buildPayload() -> api.ts -> backend -> simulationHistory -> DashboardTable
```

## 4. DashboardSidebar: apenas UI e coleta de dados

A sidebar continua tendo estado local para os valores dos inputs, mas não decide mais como montar payload nem qual endpoint chamar. Ela também não guarda mais sozinha o módulo e o query mode selecionados; essa seleção subiu para o DashboardPage.

### Estados e props principais:

```
Props recebidas:<br/>selectedModule<br/>selectedQueryMode<br/>isLoading<br/>errorMessage<br/>onModuleChange<br/>onQueryModeChange<br/>onRunSimulation<br/><br/>Estado local:<br/>fieldValues
```

Quando o usuário troca o módulo ou o modo, a sidebar chama onModuleChange ou onQueryModeChange e recria os valores iniciais dos campos para aquele contexto.

### Clique no Run Simulation:

```
const handleRunSimulation = () => {<br/>  onRunSimulation({<br/>    module: selectedModule,<br/>    queryMode: selectedQueryMode,<br/>    values: fieldValues,<br/>  });<br/>};
```

Esse trecho mostra que a sidebar não executa a simulação. Ela apenas envia os dados do formulário para o componente pai.

## 5. DashboardPage: orquestração

O DashboardPage coordena a operação. Ele sabe qual módulo e modo estão selecionados, quando a simulação está carregando, se ocorreu erro e quais chamadas já foram salvas.

### Estados principais:

```
selectedModule<br/>selectedQueryMode<br/>simulationHistory<br/>isSimulationLoading<br/>simulationError
```

### Função de execução:

```
const handleRunSimulation = async (formValues) => {<br/>  setIsSimulationLoading(true);<br/>  setSimulationError(null);<br/><br/>  try {<br/>    const result = await runSimulation(formValues);<br/>    const savedResult = {<br/>      ...result,<br/>      id: crypto.randomUUID(),<br/>      createdAt: new Date().toISOString(),<br/>    };<br/><br/>    setSimulationHistory(...);<br/>  } catch (error) {<br/>    setSimulationError(...);<br/>  } finally {<br/>    setIsSimulationLoading(false);<br/>  }<br/>};
```

Assim, a página chama a lib, salva o resultado no histórico certo e passa para a tabela apenas os dados do módulo e query mode atualmente selecionados.

## 6. simulation.ts: regra de negocio

O arquivo `src/lib/simulation.ts` concentra a regra de simulacao. Ele exporta tipos compartilhados e a funcao `runSimulation`, que e a entrada unica para executar qualquer simulacao.

### Tipos principais:

```
SimulationModuleKey = "thermo-buckling" | "thermo-fatigue" | "buckling"<br/>QueryMode =  
"Table" | "Plot Data"<br/>FieldValues = Record<string, number><br/>SimulationFormValues  
= { module, queryMode, values }<br/>SimulationResult = { module, queryMode, payload,  
data }<br/>SavedSimulationResult = SimulationResult & { id, createdAt }
```

### runSimulation:

```
export async function runSimulation({ module, queryMode, values }) {<br/> const payload  
= buildPayload(module, queryMode, values);<br/> const data = await  
postSimulation(module, queryMode, payload);<br/><br/> return { module, queryMode,  
payload, data };<br/>}
```

Essa funcao faz duas tarefas: monta o payload correto e chama o endpoint correto. O retorno inclui tambem o payload enviado, o que ajuda em debug e validacao.

## 7. Como o payload e montado

A funcao `buildPayload` usa `clonePayloadDefault` para partir sempre de um payload valido definido em `payloads.ts`. Depois ela substitui os valores do `entity` e dos parametros `X/Y` conforme o modulo e o query mode.

### Exemplo: Thermo Buckling + Table

```
const payload =  
clonePayloadDefault("thermoBucklingTable");<br/>applyEntityValues(payload.entity,  
module, values);<br/>payload.xy_pairs = [{<br/> temperature: values.temperature,<br/>  
pressure: values.pressure,<br/>}];
```

Nesse caso, o backend espera `entity` e `xy_pairs`. Por isso, os campos `temperature` e `pressure` entram dentro de `xy_pairs`.

### Exemplo: Thermo Buckling + Plot Data

```
const payload =  
clonePayloadDefault("thermoBucklingPlotData");<br/>applyEntityValues(payload.entity,  
module, values);<br/>payload.temperature_range = {<br/> start:  
values.temperature_range_start,<br/> stop: values.temperature_range_stop,<br/> step:  
values.temperature_range_step,<br/>};<br/>payload.pressure_range = {<br/> start:  
values.pressure_range_start,<br/> stop: values.pressure_range_stop,<br/> step:  
values.pressure_range_step,<br/>};
```

No `Plot Data`, o backend nao recebe `xy_pairs`. Ele recebe ranges com `start`, `stop` e `step`. Por isso a UI mostra campos de range quando o modo `Plot Data` esta selecionado.

## 8. Como o endpoint correto e escolhido

Depois que o payload esta pronto, postSimulation usa a combinacao module + queryMode para escolher a funcao correta de api.ts.

```
if (module === "thermo-buckling") {<br/> return queryMode === "Table"<br/> ?<br/> getThermoBucklingTable(payload)<br/> : getThermoBucklingPlotData(payload);<br/>}
```

A mesma logica se repete para thermo-fatigue e buckling. Dessa forma, a UI nao precisa saber qual endpoint existe. Essa decisao fica isolada na lib.

## 9. Persistencia em memoria das chamadas

A nova alteracao adiciona permanencia em memoria das chamadas feitas para a API. Em vez de guardar apenas o ultimo resultado, o DashboardPage guarda um historico separado por Simulation Module e Query Mode.

### Estrutura conceitual do historico:

```
simulationHistory = {<br/> "thermo-buckling": {<br/> Table: [],<br/> "Plot Data":<br/> [],<br/> },<br/> "thermo-fatigue": {<br/> Table: [],<br/> "Plot Data": [],<br/> },<br/> buckling: {<br/> Table: [],<br/> "Plot Data": [],<br/> },<br/> }
```

Cada chamada salva um SavedSimulationResult, que contem o resultado original mais dois metadados: id e createdAt.

### Formato de cada item salvo:

```
{<br/> id: crypto.randomUUID(),<br/> createdAt: new Date().toISOString(),<br/> module,<br/> queryMode,<br/> payload,<br/> data,<br/> }
```

O limite maximo e de 6 chamadas por combinacao de modulo e modo. Quando uma nova chamada entra, ela vai para o inicio da lista e o array e cortado com slice(0, 6).

### Trecho conceitual:

```
[<br/> savedResult,<br/> ...currentHistory[result.module][result.queryMode],<br/>].slice(0, maxSavedSimulations)
```

Isso significa que, por exemplo, Thermo Buckling + Table pode ter ate 6 chamadas salvas, enquanto Thermo Buckling + Plot Data tem outro conjunto independente de ate 6 chamadas.

## 10. Como a troca de modulo altera a tabela

Como `selectedModule` e `selectedQueryMode` agora ficam no `DashboardPage`, a pagina consegue selecionar qual parte do historico deve ser mostrada.

```
const selectedHistory =  
simulationHistory[selectedModule][selectedQueryMode];<br/><br/><DashboardTable<br/>  
results={selectedHistory}<br/> selectedModule={selectedModule}<br/>  
selectedQueryMode={selectedQueryMode}<br/>/>
```

Quando o usuario troca o modulo ou o query mode na sidebar, a tabela automaticamente recebe outro array de resultados. Assim os dados parecem persistir por modulo, porque cada modulo/modo tem seu proprio historico.

## 11. DashboardTable: lista de execucoes salvas

A tabela nao recebe mais um unico result. Ela recebe results, que e a lista das chamadas salvas para o modulo/modo selecionado.

### A tabela adiciona colunas auxiliares antes dos dados da API:

```
run: identifica a execucao salva<br/>row: identifica a linha dentro daquela  
execucao<br/>createdAt: horario em que a chamada foi salva
```

Depois disso, ela junta as linhas de todas as execucoes para mostrar uma tabela unica. Esse formato tambem cria uma base boa para o grafico, porque cada ponto pode ser associado a uma execucao especifica.

### Exemplo de resultado visual:

```
Run 1 | row 1 | createdAt | dados retornados pela API<br/>Run 1 | row 2 | createdAt |  
dados retornados pela API<br/>Run 2 | row 1 | createdAt | dados retornados pela API
```

## 12. Por que essa persistencia ajuda os graficos

Com ate 6 chamadas salvas por modulo e modo, agora e possivel transformar o historico em series para graficos. Por exemplo, cada chamada pode virar uma linha, uma barra ou um conjunto de pontos.

### Possibilidades:

- Comparar a mesma saida entre varias execucoes.
- Plotar uma serie por chamada salva.
- Criar graficos diferentes para Table e Plot Data sem misturar formatos incompativeis.
- Usar payload e createdAt para rotular execucoes no grafico.
- Permitir evoluir depois para limpar historico, renomear runs ou exportar dados.

## 13. Resumo final

A arquitetura agora tem tres camadas bem definidas: a sidebar coleta os dados, a pagina orquestra a simulacao e guarda o historico, e a lib executa a regra de payload/API. A tabela consome o historico filtrado pelo modulo e modo selecionados. Esse desenho deixa o app pronto para a proxima etapa: criar graficos a partir das chamadas persistidas em memoria.